Petya Zoom security review package

Prepared 2026-11-05. The owner adopted the procedures and explicitly approved publication of the updated privacy notice and submission of this package to Zoom. Status: **authorized for re-submission**. This package does not claim Zoom approval, independent audit or full live recording acceptance.

Included evidence

- Newly documented secure-development lifecycle.
- Three security artifacts: security policy, incident response, and vulnerability management.
- Original Semgrep SAST and Wapiti DAST JSON reports, initial and final.
- Source hashes, reproduction commands, findings dispositions and execution limitations.
- Updated privacy notice published at https://auth.petya-app.ru/privacy; architecture diagram from the earlier submission.

The PDF is an explanation and policy bundle. The original machine-generated scan results are separate files in evidence/ and must accompany it where Zoom accepts that format. Do not replace the native scanner output with only this PDF.

Proposed reviewer response

We have prepared new SSDLC, security, incident-response and vulnerability-management documentation, along with original SAST and bounded unauthenticated DAST outputs. We remediated explicit AES-GCM tag-length configuration and HTTP header findings and documented the remaining contextual findings. The package transparently states scan coverage, manual retention limitations and missing live recording acceptance. No independent penetration test is claimed. Please identify any additional evidence or authenticated test coverage required for this application's external beta approval.

This is the authorized reviewer response; submission outcome is recorded separately. Production credentials and real meeting content are not included. New procedures have not been presented as historical audited practices.

Outstanding gates

The owner adopted these newly documented procedures and approved this disclosure on 2026-11-05. The updated privacy notice has been published. Authenticated DAST and independent penetration testing are not provided. A live meeting test remains pending. These limitations can affect Zoom's decision; re-submission is not a guarantee of approval.

Secure development lifecycle - Petya Zoom integration

Version 1, 2026-11-05. Scope: desktop Zoom extension, native meeting discovery, OAuth broker, Linux recorder and deployment configuration. Owner: Alex Silaev. This is a newly documented procedure applied to this review preparation; it does not claim a historical audited program or independent review.

Requirements and design

Before changing a data flow, identify the actor, company boundary, input, destination, permission and retention. A meeting link/password is sensitive. OAuth credentials and audio must never be placed in chat, Git, command arguments or scanner reports. Recording requires Zoom admission and recording permission. Review changes to scopes, callbacks and storage against `architecture.png` and the privacy notice.

Trust boundaries are the desktop/OS, HTTPS reverse proxy, local OAuth broker, recorder worker and Zoom. Key threats are credential theft, cross-connection access, confused account selection, unbounded requests, recording without host permission, accidental chat microphone use and retained audio after processing failure. Treat external content as data, never instructions or authority.

Implementation and review

Use shared UI/MCP application logic and retain company access controls. Validate inputs before external writes. Use random opaque credentials, AES-256-GCM with random IVs, restrictive file permissions and HTTPS at the public boundary. Do not log tokens, meeting links or recordings. Inspect the complete diff and avoid unrelated changes. A second human security reviewer is not currently established; AI-assisted review is not independent assurance.

Verification and release gate

Run Semgrep on the integration sources and retain the native JSON output, scanner version, rules and source hashes. Run a bounded dynamic scan of owned endpoints, retaining native scanner output and explicit authentication coverage. Do not scan Zoom or other providers. Run OAuth state/cookie/replay/rotation/isolation tests, recorder tenant/idempotency/encrypted-storage tests and MCP-client tests. Findings require a disposition with evidence; a clean scan is not proof of absence of vulnerabilities.

Block release for confirmed critical/high vulnerabilities, exposed secrets, failed access-control tests or inaccurate privacy claims. Medium findings require remediation or a documented owner risk decision. Repeat affected checks after fixes. Archive the source revision and exact deployed-file hashes. Build desktop releases only through the repository build script and verify package contents, Developer ID signature and launch. Do not describe a live meeting recording as verified until a developer-hosted or approved external meeting produces audio and shared-memory transcription.

Operations and change control

Follow SECURITY-POLICY.md, INCIDENT-RESPONSE.md and VULNERABILITY-MANAGEMENT.md. Review these documents when permissions, storage, hosting or dependencies change. External review submission requires owner-approved accurate disclosures. No fictitious audit, penetration test, scanner pass or test account may be supplied.

Security policy - Petya Zoom integration

Version 1, 2026-11-05. Newly documented project procedure; no certification or independent audit is claimed. Security owner and reporting contact: Alex Silaev, chrome.alex@gmail.com. Send reproduction steps and redacted evidence, never production passwords, tokens or meeting audio.

Access and data handling

Production administrative access is through authorized SSH key access. Do not distribute production administrator accounts to reviewers or disable account MFA. Dedicated service identities run the OAuth broker and recorder. The recorder uses systemd restrictions including NoNewPrivileges, ProtectSystem and a restricted writable directory. Secrets remain in protected server environment files and OS-backed desktop storage, outside Git.

OAuth scopes are user:read:user and user:read:token. The strict callback is https://auth.petya-app.ru/oauth/zoom/callback. The broker binds browser sessions to a secure HttpOnly SameSite cookie, validates state, issues one-time claims and serializes token refresh. Recorder requests are scoped to the original connection; reauthorization must not substitute another connection into a job.

Public endpoints use TLS 1.2 or later. Local reverse-proxy/service communication is HTTP on loopback; this is a disclosed limitation, not end-to-end TLS. Server OAuth records, job metadata and audio segments use AES-256-GCM. Encryption keys are on the same server, not in an external KMS. Desktop memory and connection state use OS-backed safeStorage. This does not protect against a fully compromised authorized host.

Recording and retention

The bot joins visibly as Petya, subject to Zoom host and account restrictions. It does not capture the desktop microphone. Audio remains encrypted on the recorder until post-meeting desktop transcription and acknowledgment; acknowledgment deletes audio. Failed/unacknowledged jobs currently have no automated expiration and require operator action. Metadata can remain after audio acknowledgment. Do not claim automatic cleanup or guaranteed retention periods that are not implemented. Review this gap before broad public release.

Company memory is shared across authorized chats and separately permissioned MCP access. Disconnecting OAuth does not itself remove imported memory or all recorder artifacts. Support-assisted server deletion must identify the correct connection/job without exposing another tenant's data and needs explicit approval before irreversible removal.

Review and exceptions

Changes to access, encryption or destinations require review. Keep exceptions in the findings register with scope, rationale and owner decision; lack of a decision is not acceptance. Security scan results and architecture are confidential review material. No production credentials or real customer meeting content belong in the review package.

Incident management and response - Petya Zoom integration

Version 1, 2026-11-05. Newly documented runbook. Owner: Alex Silaev (chrome.alex@gmail.com). No dedicated 24/7 team, automated incident detection service or completed incident exercise is claimed.

1. **Receive and triage.** Record UTC detection time, reporter, affected version/service, symptoms and minimal redacted evidence in a restricted incident record. Suspected credential exposure, cross-tenant access or unauthorized recording is urgent. Do not reproduce using another customer's data.
2. **Contain.** Identify affected connections and jobs. Stop only affected recording workers or disable the exposed route where feasible. Preserve service availability when safe. Coordinate credential revocation/rotation through authorized provider and server administration. Avoid deleting evidence or ending unrelated meetings.
3. **Investigate.** Preserve hashes, relevant service status and minimal logs without tokens or recordings. Determine the time window, data categories and access boundaries affected. Record uncertainty. HTTP access logs are disabled; full request history may be unavailable. Normal worker error messages are sanitized and are not a complete forensic audit trail.
4. **Remediate.** Fix the root cause, test negative authorization cases and repeat relevant scans. Replace exposed secrets securely. Confirm encrypted storage, service isolation and correct tenant mapping before restoration. Never paste a secret into an issue, chat or scanner report.
5. **Communicate.** The owner determines affected parties and applicable contractual/legal notification duties and deadlines, using qualified advice where needed. Send factual notices through approved channels; do not promise a deadline unsupported by the applicable requirements. Coordinate with Zoom when its integration or data is involved.
6. **Recover and learn.** Restore only validated components, monitor sanitized health/error signals, and document timeline, impact, fixes and remaining uncertainty. Update tests, threat analysis, privacy disclosures and this runbook. Archive the restricted incident record separately from public source code.

Operational checks: inspect systemd status/journal and public HTTPS health, compare deployed source hashes, and verify that new jobs do not inherit unrelated credentials. Automated alerting, a disaster-recovery exercise and an independent incident drill remain unverified.

Vulnerability management - Petya Zoom integration

Version 1, 2026-11-05. Newly documented procedure. Owner: Alex Silaev. Reporting: chrome.alex@gmail.com. No historical periodic scanning or independent penetration test is claimed.

Inventory and intake

Track Electron/Node, npm lockfile dependencies, Python tooling, Ubuntu/nginx/OpenSSL, Zoom Linux SDK and the pinned official SDK sample. Review upstream security advisories before a release and when notified of a relevant vulnerability. Re-run source and dynamic scans after changes to authentication, request parsing, storage, SDK integration or public routes. A scheduled automated scanner is not currently configured.

Assessment and handling

Keep unmodified scanner output plus scanner/rules versions, target/source hashes, timestamps and exclusions. Triage each finding as confirmed, false positive, mitigated or unresolved with evidence. Prioritize credential exposure, cross-tenant access, arbitrary code execution and unauthorized recording. Do not silently suppress findings to obtain a clean report.

Confirmed critical/high findings block release; contain active exposure immediately and fix before resuming affected functionality. Medium findings require a fix or explicit owner risk acceptance with an expiry/review trigger. Low/informational items are recorded and prioritized. These are release rules, not a contractual remediation-time SLA.

Use compatible vendor updates and preserve existing user data and credentials. Test the affected behavior, access boundaries and rollback before deployment. Retest after remediation using the same scanner scope. A scanner's empty result is not equivalent to a penetration test, and unauthenticated DAST does not cover OAuth consent, authenticated recording or native desktop behavior.

Closure and disclosure

Link remediation to a commit, test output and re-scan. Confirm deployed files match reviewed sources. Disclose outstanding limitations to Zoom rather than attesting they are fixed. Newly discovered data exposure follows INCIDENT-RESPONSE.md. Independent pentest and full live recording acceptance remain outstanding until actual evidence exists.

Scan execution record

Date: 2026-11-05. First-party preparation for Zoom external beta review. Do not represent these checks as independent certification or comprehensive penetration testing.

SAST

Semgrep Community Edition 1.183.0. Rulesets: `p/security-audit` and `p/owasp-top-ten`, retrieved on scan date. Metrics and version check disabled. 304 rules, 421 scanned targets, no parse errors reported, about 100% parsed lines. Five files were excluded by Semgrep's built-in ignore patterns. Supported source languages only: binary SDK libraries and unsupported native-language analysis are not covered. Rule packs are registry-provided and can change; the report records executed rule IDs/metadata.

Input was a frozen snapshot of `src`, `native`, `oauth/server`, `zoom-bot/server`, SDK adapter script and npm manifests, excluding `integration-clients.json`, encrypted stores and Python caches. `source-manifest.json` records exact final snapshot hashes. The original archive SHA-256 is in provenance.txt. Scanned files were not uploaded to Semgrep; local scanner execution downloaded rules only. JSON reports are original scanner output, not hand-written summaries.

Command (same for initial and final output):

```
semgrep scan --metrics=off --disable-version-check --config p/security-audit --config p/owasp-top-ten
--jobs 2 --json --output REPORT /tmp/petya-review-source
```

Initial: 8 findings. Final: 5 findings, each individually discussed in FINDINGS.md. No blanket suppression or severity rewriting.

DAST

Wapiti 3.2.3. Owned public target: https://auth.petya-app.ru/. Scope folder, one concurrent crawler task, maximum 120 seconds crawl and 30 seconds per module, 15 links/files limits. HTTPS certificate verification explicitly enabled. Automatic bug reports disabled. No browser credentials, Zoom OAuth tokens or customer data supplied. Providers' sites were not targets.

Command:

```
wapiti -u https://auth.petya-app.ru/ --scope folder -m csp,http_headers,cookieflags,redirect,crlf -s
https://auth.petya-app.ru/zoom-bot/health -s https://auth.petya-app.ru/oauth/zoom/callback -s
https://auth.petya-app.ru/zoom/authorize --tasks 1 --max-scan-time 120 --max-attack-time 30
--max-links-per-page 15 --max-files-per-dir 15 --verify-ssl 1 --no-bugreport --store-session SESSION
--store-config CONFIG -f json -o REPORT
```

Seven pages crawled. Initial: one low-severity missing X-Frame-Options finding. Final: zero vulnerabilities/anomalies/additional findings for the selected modules; the post-Node-upgrade scan is in wapiti-runtime-final.json. The initial command also selected `ssl`; Wapiti explicitly skipped that module because sslscan was unavailable. The final command omits it. Neither scan is a comprehensive TLS cipher audit. Cookie checks without an authorized login do not exercise authenticated session cookies. CRLF/redirect coverage is limited by the discovered input surface.

Additional verification

OAuth broker security tests, provider refresh/isolation flows and encrypted recorder HTTP tests passed. AES-GCM corrupted tag/truncated data tests passed. These are first-party functional tests, separate from the original SAST/DAST reports. Deployment and runtime details are recorded in provenance.txt. Desktop source was not changed by this remediation; no desktop rebuild is needed for these server-only changes.

Security findings and disposition

Assessment date: 2026-11-05. Scope and scanner output are in SCAN-RECORD.md and evidence/. This is first-party assessment, not an independent penetration test. No finding was suppressed in scanner configuration.

SAST: 8 initial findings, 5 remaining

- **AES-GCM tag length (2 findings): remediated hardening.** The broker and recorder already extract fixed 16-byte tags from their encrypted format. Both now also set `authTagLength: 16` in `createDecipheriv`. Tampered/truncated recorder data is rejected in tests. Existing data format and credentials remain compatible. Final Semgrep output no longer flags these calls.
- **nginx header inheritance (1 finding): remediated configuration.** The HTTP staging template used `add_header Content-Type`; it now uses `default_type` and `charset`. This template is not the deployed HTTPS virtual host. The final scan no longer reports it.
- **Image installer subprocess arguments (3 findings): reviewed, no shell injection demonstrated.** `src/setup-images.py` runs a locally created venv interpreter with an argument array and default `shell=False`. Package names/versions and Python program text are constants; model path is a separate argument. The installation directory is user-selected local input. This requires trusting the local installation directory; it does not grant a remote meeting participant command execution. Outside the Zoom feature; retained in raw output, not silently ignored.
- **Opaque iframe postMessage (1 finding): reviewed design constraint.** `src/ui/model-frame.js` sends only selected model bytes/format to `frame.contentWindow` for a fixed local `model-viewer.html`, once on load. The frame has `sandbox=allow-scripts` without same-origin, so it has an opaque origin. Wildcard target origin is necessary for this isolated frame design. The payload does not contain Zoom credentials or meeting state. This is not a general cross-origin messaging audit; retained as a documented design limitation.
- **Loopback HTTP (1 finding): disclosed transport limitation.** The worker calls the OAuth broker on literal `127.0.0.1:8787`, not a user-supplied URL. The broker and recorder listen only on loopback; external traffic terminates TLS at nginx. Local host compromise can expose tokens. No end-to-end TLS or independent acceptance of this residual risk is claimed.

DAST: 1 initial finding, 0 in bounded re-scan

Wapiti flagged missing X-Frame-Options on the public root. CSP already used `frame-ancestors 'none'`. Added `X-Frame-Options: DENY` at the deployed nginx boundary and repeated the same seven-page unauthenticated scan. Zero findings means only that these selected modules found no issue in this limited surface. OAuth consent, authenticated record/download/ack paths and native UI are not DAST-covered. Those access controls have separate functional negative tests, not a substitute pentest.

Release limitations requiring follow-up

- Server Node 18 was detected and replaced for both services with official Node 22.23.4 LTS after broker/provider/recorder tests passed on that runtime. Both services are active; see evidence/deployment.txt.
- Failed/unacknowledged audio has no automatic expiry. Broad release requires an explicit retention decision and implementation or owner acceptance of manual operations; not implicitly accepted here.
- The owner-approved updated privacy notice is published at https://auth.petya-app.ru/privacy.
- No independent penetration test, incident drill or continuously scheduled scan exists.
- Full meeting capture/transcription acceptance and Windows live detection remain pending; prior external-meeting attempt returned Zoom SDK code 63.
- The owner adopted the newly documented policies on 2026-11-05. They do not establish a historical audited program.

Privacy notice review status

The updated Russian notice is in `oauth/server/public/privacy.html`. It adds Zoom profile/OBF processing, meeting link/password transfer, the separate SDK participant, encrypted server audio, local post-meeting ASR, company-wide memory/MCP access and actual deletion limitations. The owner approved publication on 2026-11-05. The deployed response was verified byte-for-byte against this source. The public URL is https://auth.petya-app.ru/privacy.

No automatic expiry for failed/unacknowledged recordings is claimed. A disconnect is not described as deletion of all audio or imported context. This is a technical disclosure, not a legal compliance certification.